

Pour tester toutes vos fonctions, la fonction `main` de `TP28.c` contient une recherche de recherche "u moment où l" dans le texte intégral de la Recherche du Temps Perdu : il doit apparaître 30 fois. Le programme affiche également le temps que cette recherche a pris.

Dans tout le sujet, on considère `s` motif de longueur m et `t` texte de longueur n .

Exercice 1 :

Algorithme naïf et algorithme de Boyer-Moore

1. Écrire une fonction `int recherche_naive(char* s, char* t)` qui compte le nombre d'occurrences d'un motif `s` dans un texte `t`, en partant de l'indice $i = 0$ dans le texte et en essayant de faire correspondre à `s` à partir de la gauche. Si une correspondance est trouvée, ou bien s'il n'existe aucune correspondance, on essaye à l'indice $i + 1$.
2. On veut maintenant implémenter l'algorithme de Boyer-Moore :
 - (a) Écrire une fonction `int** construire_table(char* s)` qui étant donné un motif `s` de longueur m , construit la matrice `table` de taille $m \times 256$ telle que pour tout indice $0 \leq j < m$ et tout caractère c , $k = \text{table}[j][c]$ est le plus grand indice de $\llbracket 0; j-1 \rrbracket$ tel que $s[k] = c$ si un tel indice existe, et est -1 sinon.
 - (b) En déduire une fonction `int boyer_moore(char* s, char* t)` qui compte encore le nombre d'occurrences d'un motif dans un texte, mais cette fois-ci en utilisant l'algorithme de Boyer-Moore.
3. (a) Quels sont les complexités, en fonction de m et n , de l'algorithme naïf et de l'algorithme de Boyer-Moore dans le pire cas ? Le meilleur cas ? Un cas "typique" ?
On pourra supposer n grand devant m , et donc en particulier $n > 2m$.
- (b) Compte tenu de la taille du texte et du motif qu'on a utilisés, et du fait qu'on est dans un cas "normal", que peut-on dire du rapport entre les temps d'exécutions de l'algorithme de Boyer-Moore et de l'algorithme naïf ?

Exercice 2 :

Algorithme de Rabin-Karp

L'algorithme de Rabin-Karp consiste à se donner une fonction de hachage h et, pour chaque $i \in \llbracket 0; n-m \rrbracket$, comparer $h(s)$ et $h(t[i \dots (i+m-1)])$.

Si les hashés sont différents, alors les antécédents `s` et `t[i ... (i+m-1)]` sont différents, donc il n'y a pas d'occurrence de `s` à l'indice i . Si ils sont égaux, ceci ne prouve pas l'existence d'une occurrence : il peut aussi s'agir d'une collision pour la fonction de hachage, et on vérifie de façon naïve si `s = t[i ... (i+m-1)]`.

1. On suppose que pour chaque i , $h(t[i \dots (i+m-1)])$ se calcule en coût constant indépendant de m . Donner la complexité dans le meilleur et dans le pire cas de l'algorithme de Rabin-Karp dans ce cas. Quel vous semble être un cas où l'algorithme de Rabin-Karp pourrait être préférable à celui de Boyer-Moore ?
2. On se donne $a = 256$ et p premier impair. Pour toute chaîne de caractères `t` de longueur au moins k , on pose $h_{a,p}(t[0 \dots (k-1)]) = \left(\sum_{i=0}^{k-1} t[i] a^{k-i-1} \right) \bmod p$.

La fonction `int hash(int a, int p, char* t, int k)` dans le fichier `TP28.c` permet de calculer cette fonction.

- (a) Quelle est la complexité en fonction de m du calcul de $h_{a,p}(t[i \dots (i+m-1)])$ à partir des données de `t`, $i \in \llbracket 0; n-m \rrbracket$ et m ?
 - (b) Écrire une fonction qui, en utilisant a , p , `t`, $i \in \llbracket 1; m-n \rrbracket$, m et $h_{a,p}(t[(i-1) \dots (i+m-2)])$, calcule $h_{a,p}(t[i \dots (i+m-1)])$ en temps constant.
Quel précalcul faut-il faire (et fournir à cette fonction) ?
On dit que $h_{a,p}$ est une fonction de hachage roulante.
3. Écrire une fonction `int rabin_karp(char* s, char* t)` qui compte le nombre d'occurrences d'un motif dans un texte en utilisant l'algorithme de Rabin-Karp, avec $a = 256$ et $p = 101$.